

可行性分析报告

1. 技术可行性

1.1 技术栈成熟度

层级	技术	可行性说明
后端	Python / FastAPI / uvicorn	生态成熟 适合快速构建 REST API
分析	radon / lizard / bandit / pip-audit	覆盖复杂度 代码规模 安全漏洞和依赖风险
前端	React / TypeScript / Vite / ECharts	支持复杂交互和可视化报告
存储	SQLite / JSON 兼容策略	轻量 零配置 适合课程项目和单机部署
CI/CD	GitHub Actions / GitLink DevOps	可自动验证 lint test build
规范	OpenSpec	适合能力变更记录和需求追踪

结论 技术栈均为成熟开源方案 项目技术可行

1.2 关键技术风险

风险	影响	应对
GitHub 网络不稳定	克隆失败或耗时过长	支持代理 超时 错误提示 缓存
大仓库分析慢	用户等待时间长	异步任务 状态轮询 缓存命中
LLM API 不可用	AI 诊断失败	无 Key 本地规则诊断降级
多语言分析复杂	分析准确度不一致	先覆盖 Python/JS/TS 逐步扩展

2. 经济可行性

项目主要基于开源技术构建 不依赖付费基础设施 可选成本来自

- LLM API 调用费用
- 云服务器或部署平台费用
- GitHub Actions/GitLink 构建资源

课程实践和本地演示场景可零成本运行 经济可行

3. 进度可行性

项目采用 capability 迭代方式推进 已经完成 cap-01 到 cap-20 的能力归档 覆盖从项目初始化 分析引擎 报告 AI 前端改版 安全加固 CI/CD 到报告体验增强的完整链条

进度结论 当前版本已经达到课程答辩和演示要求

4. 操作可行性

用户操作路径简单

1. 双击一键启动脚本
2. 输入 GitHub 仓库 URL
3. 等待进度条完成
4. 查看报告 导出 分享或对比

系统对无 Key 缓存命中 网络失败 输入错误均有提示 操作可行

5. 法律与合规可行性

- 项目遵循开源依赖许可证使用方式
- 不保存用户输入的 API Key 到后端
- GitHub OAuth 仅用于登录扩展 不强制使用
- 公开仓库分析不涉及私有代码泄露

结论 在只分析公开仓库的前提下合规风险较低

6. 综合结论

Repo Health Check 在技术 经济 进度 操作和合规方面均具备可行性 适合作为开源软件课程实践项目 也具备后续扩展为真实开源质量评估工具的潜力